```
invoice-ai-system/
├── app/
│   └── main.py                 # Lance le serveur FastAPI
│
├── domain/                     # CŒUR DU PROJET (aucune dépendance technique)
│   ├── entities/
│   │   ├── invoice.py
│   │   └── invoice_item.py
│   └── repositories/
│       └── invoice_repository.py
│
├── application/                # LOGIQUE MÉTIER (Use Cases)
│   ├── use_cases/
│   │   ├── upload_invoice.py
│   │   ├── preprocess_invoice.py
│   │   ├── extract_text_ocr.py
│   │   ├── extract_fields_llm.py
│   │   ├── validate_invoice.py
│   │   └── export_invoice.py
│   └── services/
│       └── ocr_service.py      # ← Ici on changera Tesseract → Mistral
│
├── infrastructure/             # TOUT CE QUI EST TECHNIQUE (Azure, DB, stockage)
│   ├── database/
│   │   ├── postgres.py
│   │   └── models.py           # Tables SQLAlchemy
│   ├── storage/
│   │   └── file_storage.py
│   ├── ocr/
│   │   ├── tesseract_ocr.py    # Version temporaire
│   │   └── azure_ocr.py        # Version finale (mistral-document-ai-2512)
│   └── embedding/
│       └── embed_service.py    # embed-v-4-0 pour recherche
│
├── interface/                  # API (ce que voit l'utilisateur)
│   └── api/
│       ├── routers/
│       │   └── invoice_router.py
│       └── schemas/
│           └── invoice_schema.py
│
├── frontend/                   # React (optionnel pour l'instant)
│
├── tests/                      # Tests unitaires (très important pour jury)
├── requirements.txt
├── .env
└── README.md
```

app/

- **main.py** → Démarre le serveur API (comme un "bouton marche/arrêt").

---

domain/

- **entities/invoice.py** → Définit ce qu'est une facture (fournisseur, date, total).
- **entities/invoice_item.py** → Définit les lignes de facture (produit, quantité, prix).
- **repositories/invoice_repository.py** → Règles pour sauvegarder/charger les factures (sans détails techniques).

---

application/

- **use_cases/upload_invoice.py** → Gère le téléchargement d'une facture.
- **use_cases/preprocess_invoice.py** → Nettoie l'image (redressement, éclaircissement).
- **use_cases/extract_text_ocr.py** → Extrait le texte brut de l'image (via OCR).
- **use_cases/extract_fields_llm.py** → Utilise l'IA pour extraire les données (fournisseur, date, etc.).
- **use_cases/validate_invoice.py** → Permet de valider/modifier les données extraites.
- **use_cases/export_invoice.py** → Génère des fichiers CSV/JSON.
- **services/ocr_service.py** → Choix entre Tesseract (gratuit) ou Azure OCR (puissant).

---

infrastructure/

- **database/postgres.py** → Connexion à la base de données PostgreSQL.
- **database/models.py** → Structure des tables dans la base de données.
- **storage/file_storage.py** → Sauvegarde les fichiers (ex: images de factures).
- **ocr/tesseract_ocr.py** → OCR temporaire avec Tesseract (gratuit).
- **ocr/azure_ocr.py** → OCR final avec Azure (IA puissante).
- **embedding/embed_service.py** → Permet de chercher des factures par mots-clés (ex: "Microsoft").

---

interface/

- **api/routers/invoice_router.py** → Endpoints API (ex: /upload, /invoices).
- **api/schemas/invoice_schema.py** → Vérifie que les données sont au bon format (ex: date valide).

---

frontend/

- Dossier pour l'interface utilisateur (React), **optionnel** (peut être remplacé par Swagger).

Fichiers racine

- **.env** → Stocke les secrets (mot de passe de la base de données, clés API).
- **requirements.txt** → Liste des outils à installer (ex: fastapi, sqlalchemy).
- **README.md** → Mode d'emploi du projet (comment lancer, installer).
- **tests/requirements.txt** → Outils pour tester le code.

Résumé en 1 phrase

- **domain/** → Ce qu'est une facture (cœur métier).
- **application/** → Ce qu'on fait avec les factures (télécharger, extraire, valider).
- **infrastructure/** → Outils techniques (base de données, OCR, stockage).
- **interface/** → Ce que voit l'utilisateur (API, frontend).

Exemple concret :
Quand tu uploades une facture :

1. **application/use_cases/upload_invoice.py** reçoit le fichier.
2. **infrastructure/storage/file_storage.py** le sauvegarde.
3. **application/use_cases/preprocess_invoice.py** le nettoie.
4. **infrastructure/ocr/azure_ocr.py** lit le texte.
5. **application/use_cases/extract_fields_llm.py** extrait les données avec l'IA.

Mon travaille

main.py	lancer application + connecter routes + créer tables DB

application/

fichier	Role
export_invoice.py	Permettre d'exporter les factures : • en CSV (Excel) • en JSON (API)
extract_fields_llm.py	OCR → data (Transformer **texte OCR ➜ données structurées)**
get_invoice_history.py	Afficher **historique des factures**
preprocess_invoice.py	Améliorer image avant OCR (important)
search_invoices.py	Rechercher factures
submit_feedback.py	corrections utilisateur(amélioration systéme)
validate_invoice.py	confirmer données

infrastructure/

fichier	rôle
database.py	• Gérer la **connexion à la base de données (SQLite)** • Fournir une session DB utilisable dans tout le projet
models.py	définir structure table invoices
tesseract_ocr.py	transformer image → texte
file_storage.py	Sauvegarder fichiers uploadés

interface/

fichier	rôle
auth_router.py	• Gérer **authentification (login + token JWT)** • Sécuriser ton système
invoice_router.py	gérer tout le système factures (upload, OCR, extraction, recherche, export…)

سؤال	fichier
وين يتحفظ الملف ؟	file_storage
كيفاش نقرا الصورة ؟	preprocess
كيفاش نخرج texte ؟	OCR
كيفاش نخرج données ؟	extract_fields
وين نخزن ؟	database/models
كيفاش نعرض ؟	router

"السيستام هذا يخدم كيما موظف"

1. المستخدم يبعث facture
2. السيستام يقراها (OCR)
3. يخرج المعلومات
4. يخزنها
5. يتجم يقلب فيها / يخرجها / يصلحها

💡 هذا أهم حاجة تفهمها قبل الكود

```
User upload facture
        ↓
file_storage.py (يحفظ الملف)
        ↓
preprocess_invoice.py (ينظف الصورة)
        ↓
tesseract_ocr.py (يخرج texte)
        ↓
extract_fields_llm.py (يخرج données)
        ↓
database.py + models.py (يخزن)
        ↓
invoice_router.py (يعرض للمستخدم)
```

👉 جاوب على سؤال كل fichier: